

三元表达式：三元表 + 间接码表。

就是三元表 set 表 (一个指令 pool)

e.g. $X := (A+B) \times C$

$Y := D \uparrow (A+B)$

间接码	op	arg1	arg2
(1)	(1) +	A	B
(2)	(2) *	(1)	C
(3)	(3) :=	X	(2)
(4)	(4) ↑	D	(1)
(5)	(5) :=	Y	(4)

比较

三元：用了 pointer，优化得为那

间接：改间接表即可，节省空间

四元：好改，但临时变量占空间。

作为条件控制的布尔式翻译。

条件语句：if E then S_1 else S_2 .

e.g. $\text{if } a > c \text{ or } b < d \text{ then } S_1 \text{ else } S_2$

if $a > c$ goto L2

goto L1 $\rightarrow$ 真出口

L1: if $b < d$ goto L2

goto L3 $\rightarrow$ 假出口

L2: (S_1 代码序列)

goto Lnext

L3: (S_2 代码序列)

Lnext:

语义规则

$E \rightarrow id_1 \text{ relop } id_2$ E.code = gen('if' id_1 .place
relop.op id_2 .place 'goto' E.true) ||
gen('goto' E.false)

$E \rightarrow true$ E.code = gen('goto' E.true)

$E \rightarrow false$ E.code = gen('goto' E.false)

布尔表达式翻译。

数值表示法。

emit: 三地址代码输出文件中

nextstat: 给出输出序列中下一条三地址语句地址序号。

每条三地址语句，emit 把 nextstat + 1

e.g. $a < b$, 起始地址 100 (nextstat = 100)

临时变量

$E \rightarrow E_1 \text{ or } E_2$ { E.place = newtemp;
emit(E.place := 'E₁.place 'or'
E₂.place) }
 $E \rightarrow E_1 \text{ and } E_2$ { E.place = newtemp;
emit(E.place := 'E₁.place 'and'
E₂.place) }
 $E \rightarrow \text{not } E_1$ { E.place = newtemp;
emit(E.place := 'not' E₁.place) }
 $E \rightarrow (E_1)$ { E.place = E₁.place }
 $E \rightarrow id_1 \text{ relop } id_2$ { E.place = newtemp;
emit('if' id_1 .place relop.op
 id_2 .place 'goto' nextstat+3);
emit(E.place := '0');
emit('goto' nextstat+2);
emit(E.place := '1') }
 $E \rightarrow id$ { E.place = id.place }

100: if $a < b$ goto 103 (nextstat = 101)

101: $T_1 = 0$ (nextstat = 102)

102: goto 104

103: $T_1 = 1$.

104:

e.g. $a < b \text{ or } c < d \text{ and } e < f$ (100)

$E \rightarrow id_1 \text{ relop } id_2$ { E.place = newtemp;
emit('if' id_1 .place relop.op id_2 .place
'goto' nextstat+3);
emit(E.place := '0');
emit('goto' nextstat+2);
emit(E.place := '1') }
 $E \rightarrow id$ { E.place = id.place }
 $E \rightarrow E_1 \text{ or } E_2$ { E.place = newtemp;
emit(E.place := 'E₁.place 'or'
E₂.place) }
 $E \rightarrow E_1 \text{ and } E_2$ { E.place = newtemp;
emit(E.place := 'E₁.place 'and'
E₂.place) }

106: goto 108

107: $T_2 = 1$

108: if $e < f$ goto 111

109: $T_3 = 0$

110: goto 112

100: if $a < b$ goto 103

101: $T_1 = 0$

102: goto 104

103: $T_1 = 1$

104: if $c < d$ goto 107

105: $T_2 = 0$

111: $T_3 = 1$

112: $T_4 = T_2 \text{ and } T_3$

113: $T_5 = T_1 \text{ or } T_4$

布尔表达式两遍扫描。

构造语法树

DFS, 递归写

布尔表达式一遍翻译。

四元式

约定:

(jnz, a, -, p): if a goto p

(jrop, x, y, p): if $x \text{ rop } y$ goto p

(j, -, -, p): goto p.

有地址地址不一定知道，要两遍，E 位置。

E.true list, E.false list.

nextquad: 初值为 1, 执行一项 emit, nextquad++

makelist(i): 创建链表，返回指向链表的头。

merge(P_1, P_2): P_1 链表, P_2 链表合一起，返回合并链表

backpatch(p, t): 回填，把 p 链接到的那个四元式

4th segment 填为 t

产生式	语义规则
$E \rightarrow E1 \text{ or } M E2$	{ backpatch(E1.falselist, M.quad); E.truelist:=merge(E1.truelist, E2.truelist); E.falselist:=E2.falselist }
$E \rightarrow E1 \text{ and } M E2$	{ backpatch(E1.truelist, M.quad); E.truelist:=E2.truelist; E.falselist:=merge(E1.falselist, E2.falselist) }
$E \rightarrow (E1)$	{ E.truelist:=E1.truelist; E.falselist:=E1.falselist }
$E \rightarrow id1 \text{ relop } id2$	{ E.truelist:=makelist(nextquad); E.falselist:=makelist(nextquad+1); emit('j' relop.op ' ' id1.place ' ' id2.place ' ' '0'); emit('j, -, -, 0') }
$E \rightarrow id$	{ E.truelist:=makelist(nextquad); E.falselist:=makelist(nextquad+1); emit('jnz' ' ' id.place ' ' '-' ' ' '0'); emit('j, -, -, 0') }
$S \rightarrow \text{if } E \text{ then } M1 \text{ } S1 \text{ } N \text{ else } M2 \text{ } S2$	{backpatch(E.truelist, M1.quad); backpatch(E.falselist, M2.quad); S.nextlist:=merge(S1.nextlist, S2.nextlist) }
$S \rightarrow \text{while } M1 \text{ } E \text{ do } M2 \text{ } S1$	{backpatch(S1.nextlist, M1.quad); backpatch(E.truelist, M2.quad); S.nextlist:=E.falselist emit('j, -, -, ' M1.quad) }
$S \rightarrow id:=E$	{ emit(':=' ' ' E.place ' ' '-' ' ' id.place) }
$E \rightarrow E1 \text{ op } E2$	{ E.place:=newtemp; emit(op ' ' E1.place ' ' E2.place ' ' E.place) }
$M \rightarrow \varepsilon$	{ M.quad:=nextquad }
$N \rightarrow \varepsilon$	{ N.nextlist:=makelist(nextquad); emit('j, -, -, 0') }

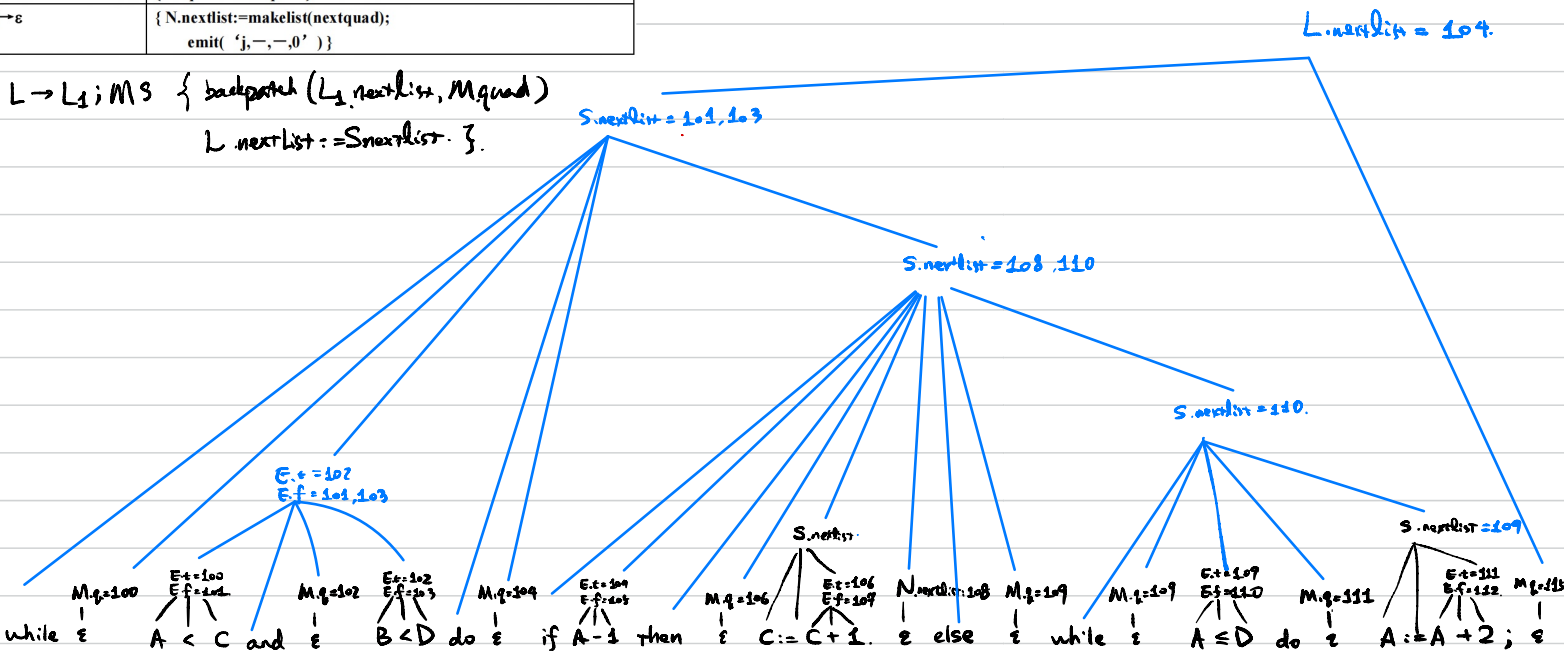
<e.g.> 四元式:

while $A < C$ and $B < D$ do

if $A = 1$ then $C := C + 1$ else

while $A \leq D$ do $A := A + 2$;

$L \rightarrow L_1; M S \quad \{ \text{backpatch}(L_1.\text{nextlist}, M.\text{quad})$
 $L.\text{nextlist} := S.\text{nextlist}; \}$



产生式	语义规则
$E \rightarrow E1 \text{ or } M E2$	{ backpatch(E1.falselist, M.quad); E.truelist:=merge(E1.truelist, E2.truelist); E.falselist:=E2.falselist }
$E \rightarrow E1 \text{ and } M E2$	{ backpatch(E1.truelist, M.quad); E.truelist:=E2.truelist; E.falselist:=merge(E1.falselist, E2.falselist) }
$E \rightarrow (E1)$	{ E.truelist:=E1.truelist; E.falselist:=E1.falselist }
$E \rightarrow id1 \text{ relop } id2$	{ E.truelist:=makelist(nextquad); E.falselist:=makelist(nextquad+1); emit('j' relop.op ' ' id1.place ' ' id2.place ' ' '0'); emit('j, -, -, 0') }
$E \rightarrow id$	{ E.truelist:=makelist(nextquad); E.falselist:=makelist(nextquad+1); emit('jnz' ' ' id.place ' ' '-' ' ' '0'); emit('j, -, -, 0') }
$S \rightarrow \text{if } E \text{ then } M1 \text{ } S1 \text{ } N \text{ else } M2 \text{ } S2$	{backpatch(E.truelist, M1.quad); backpatch(E.falselist, M2.quad); S.nextlist:=merge(S1.nextlist, S2.nextlist) }
$S \rightarrow \text{while } M1 \text{ } E \text{ do } M2 \text{ } S1$	{backpatch(S1.nextlist, M1.quad); backpatch(E.truelist, M2.quad); S.nextlist:=E.falselist emit('j, -, -, ' M1.quad) }
$S \rightarrow id:=E$	{ emit(':=' ' ' E.place ' ' '-' ' ' id.place) }
$E \rightarrow E1 \text{ op } E2$	{ E.place:=newtemp; emit(op ' ' E1.place ' ' E2.place ' ' E.place) }
$M \rightarrow \varepsilon$	{ M.quad:=nextquad }
$N \rightarrow \varepsilon$	{ N.nextlist:=makelist(nextquad); emit('j, -, -, 0') }

↓ 逆序: 在归约过程中
 序2

100 (j<, A, C, 102)

101 (j, -, -, 105)

102 (j<, B, D, 104)

103 (j, -, -, 105)

104 (j=, A, 1, 106)

105 (j, -, -, 109)

106 (+, C, 1, T1)

107 (:= T1, -, C)

108 (j, -, -, 100) $N \rightarrow \varepsilon$ emit 序2.

109 (j≤, A, D, 111)

110 (j, -, -, 100)

111 (+, A, 2, T2)

112 (:= T2, -, A)

113 (j, -, -, 109)

114 (j, -, -, 100)

115

小while 序2.

大while 序2

这回就正常了!

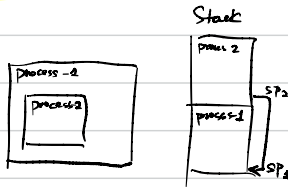
。层次：主：0 调用的过程， $i+1$ 。

。静态链 & 活动记录。嵌套层次表 display。

★ 静态链：活动记录。

。静态链：

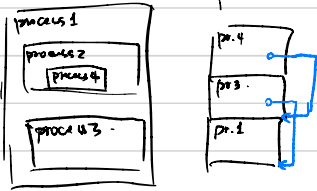
嵌套关系。



存取链

。动态链

调用关系。



控制链

$Pr.1 \rightarrow Pr.3 \rightarrow Pr.4$

★ Summary.

general:

local variables

local display.

开始

#params.

old top

global display.

net. Addr.

old sp.

$sp \rightarrow TOP(now)$

main:

local variables

global Display.

$SP_0 \rightarrow TOP(main)$

$I \leftarrow 0$

previous SP_3

★ DISPLAY

内容：Program P

Procedure Q

Procedure R

。嵌套层次显示表

R的DISPLAY: 2 R-sp.

1 Q-sp

0 P-sp

。嵌套过程活动记录

